ZoonoseApp
Plano CTO — MVP Completo

v1.0 — Março 2026

DOCUMENTO TÉCNICO ESTRATÉGICO

Plano CTO: MVP ZoonoseApp

Tudo que os founders precisam saber para contratar, executar e demonstrar o MVP em 4–8 semanas. Stack, modelagem, backlog, cronograma, o que fazer sozinhos e o que terceirizar.

- Stack: Expo + Supabase + Next.js
- Prazo: 4–8 semanas
- Budget: R$ 8.000–15.000
- 14 blocos estratégicos

ÍNDICE — 14 BLOCOS

01 Decisão Central da Stack
02 Definição do MVP
03 Arquitetura do MVP
04 Modelagem do Sistema
05 Fluxos e Telas
06 Passo a Passo de Execução
07 Backlog Técnico Atômico
08 O que Founders Fazem Sozinhos
09 O que Vale Terceirizar
10 Cronograma
11 Prompts para Subagentes
12 Critérios de Demo
13 O que NÃO Fazer Agora
14 Recomendação Final

01 Decisão Central da Stack

A stack escolhida, o que é software, o que é manual no início e o que fica completamente fora do MVP.

STACK ESCOLHIDA

Expo

APP DO AGENTE (ANDROID)

Supabase

BANCO + AUTH + STORAGE

Next.js

PAINEL COORDENADOR (WEB)

💻 É SOFTWARE	✎ É MANUAL NO INÍCIO	❌ FICA FORA AGORA
Login / autentição por e-mail e senha	Seed de imóveis (importar CSV manualmente)	Mapas e heatmaps de densidade
Cadastro e listagem de imóveis	Notificações (WhatsApp manual)	Roteirização de visitas
Registro de visita com 6 status	Relatórios (exportar CSV + abrir no Sheets)	App iOS
Foto / evidência por visita	Onboarding de agentes ao vivo	Push notifications automáticas
Modo offline + sincronização automática	Cobrança via invoice manual (Stripe depois)	Multi-tenant (múltiplas prefeituras)
Painel de cobertura e pendências	—	IA, analytics avançados, billing integrado
Exportação CSV	—	—

⏰ **Menor Caminho Estimado até Demo**

Schema: 1 dia → Scaffold + auth: 1 dia → App do agente (8 telas): 2 semanas → Painel coordenador: 1 semana → Seed + demo: 3 dias

Total: 4–5 semanas com um dev dedicado.

02 Definição do MVP

O que entra, o que adiamos e o que cortamos completamente. Cada decisão justificada.

Critério de Corte

"Se não ajuda a prefeitura a entender o valor em 20 minutos de demo, não entra."

✅ FUNCIONALIDADES INDISPENSÁVEIS (11 ITENS)

FUNCIONALIDADE	MOTIVO DE ENTRAR	DIFICULDADE	FOUNDERS FAZEM?	TERCEIRIZAR?
Login e-mail/senha	Primeiro passo de qualquer demo. Sem isso, nada funciona.	BAIXO	NÃO	SIM
Perfis por role (agente, coordenador, admin)	Mostra que o sistema é multi-usuário e seguro por design.	MÉDIO	NÃO	SIM
Cadastro de imóvel via CSV	Prefeitura já tem planilha. Importar é o que valida integração.	BAIXO	SIM	PARCIAL
Registro de visita com 6 status	Núcleo do produto. Sem registro de visita não há ZoonoseApp.	MÉDIO	NÃO	SIM
Foto / evidência	Prova para coordenador. Diferencial vs. papel.	MÉDIO	NÃO	SIM
Offline + sync MAIOR RISCO	Agentes trabalham sem internet. Sem isso o app não é usável no campo.	CRÍTICO	NÃO	SIM

FUNCIONALIDADE	MOTIVO DE ENTRAR	DIFICULDADE	FOUNDERS FAZEM?	TERCEIRIZAR?
Histórico do imóvel	Mostra reincidência. Argumento forte para prefeitura.	BAIXO	NÃO	SIM
Painel do coordenador	A "tela de WOW" da demo. Cobertura + pendências em tempo real.	ALTO	NÃO	SIM
Exportação CSV	Prefeitura quer dados no Excel. CSV é o mínimo para isso.	BAIXO	NÃO	PARCIAL
Admin criar usuários	Para o piloto real, o coordenador precisa adicionar agentes.	BAIXO	NÃO	SIM
Seed de dados de demo	Demo sem dados parece protótipo. Dados reais de Uberlândia convencem.	BAIXO	SIM	NÃO

⏳ ADIADOS PARA V2

- 📍 Mapa e heatmap de ocorrências
- 🔔 Push notifications automáticas
- 🚚 Roteirização de visitas
- 📄 Relatórios PDF automáticos
- 🍓 App iOS (App Store)
- 🏢 Multi-tenant (várias prefeituras)

❌ Cortado Definitivamente do MVP

IA e machine learning

Analytics avançados (dashboards complexos)

Billing integrado (Stripe, pagamentos)

Auditoria automática de campo

Integração com sistemas legados municipais

03 Arquitetura do MVP

10 camadas, por que cada uma foi escolhida e qual o trade-off aceito.

CAMADA	FERRAMENTA	POR QUE ESCOLHIDA	ALTERNATIVA	TRADE-OFF ACEITO	DIFICULDADE
App do Agente	Expo (React Native)	Android first, TypeScript, comunidade enorme, EAS Build para APK sem Mac	Flutter	Bundle maior, JS bridge overhead	MÉDIO
Painel Coordenador	Next.js + Tailwind CSS	SSR opcional, Vercel deploy em 1 clique, ecossistema React compartilhado com Expo	Vite + React	Overhead Next.js para app simples	MÉDIO
Banco de Dados	Supabase PostgreSQL	SQL real, RLS nativo, interface visual, plano free genéroso para MVP	Firebase Firestore	Vendor lock-in leve no Supabase	BAIXO
Autenticação	Supabase Auth	Zero config, JWT nativo, integrado com RLS, e-mail/senha pronto	Auth0, Clerk	Customização limitada de UI	BAIXO
Storage de Fotos	Supabase Storage	Integrado ao mesmo projeto, URLs públicas ou privadas, policies via SQL	Cloudinary, S3	Limitação de 1GB no plano free	BAIXO

CAMADA	FERRAMENTA	POR QUE ESCOLHIDA	ALTERNATIVA	TRADE-OFF ACEITO	DIFICULDADE
Offline / Sync	MMKV + fila local	MMKV é o storage mais rápido para React Native, fila simples de operações pendentes	WatermelonDB, SQLite	Sem conflict resolution avançado	CRÍTICO
Histórico do Imóvel	Tabela visits (FK property)	Query SQL simples, sem complexidade adicional, join nativo	Tabela separada de audit	Histórico só de visitas, não de alterações de campo	BAIXO
Exportação	Query SQL + CSV no servidor	Simples, confiável, abre no Excel sem problemas de encoding	Biblioteca CSV no client	Não é real-time, gera sob demanda	BAIXO
Controle de Acesso	Supabase RLS	Policies no banco, impossibível bypass pelo client, sem middleware extra	Middleware Node.js	Curva de aprendizado SQL policies	MÉDIO
Build APK	EAS Build (Expo)	Build na nuvem, não precisa de Mac nem Android Studio, APK em 20 min	Build local	Dependência do serviço Expo cloud	BAIXO

04 Modelagem do Sistema

6 entidades, SQL completo e tabela de permissões por role.

6 Entidades do MVP

`profiles` — `sectors` — `properties` — `visits` — `evidences` — `activity_logs`

```
-- ========================================
-- ZOONOSEAPP MVP — SCHEMA COMPLETO
-- ========================================
```

```
-- 1. SETORES (bairros / regiões de cobertura)
CREATE TABLE sectors (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  municipality TEXT NOT NULL DEFAULT 'Uberlândia',
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- 2. PERFIS DE USUÁRIO (extende auth.users)
CREATE TABLE profiles (
  id         UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  name       TEXT NOT NULL,
  role       TEXT NOT NULL CHECK (role IN ('agent','coordinator','manager','admin')),
  sector_id  UUID REFERENCES sectors(id),
  active     BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. IMÓVEIS
CREATE TABLE properties (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code       TEXT UNIQUE NOT NULL,   -- ex: UBL-0042
  address    TEXT NOT NULL,
  sector_id  UUID REFERENCES sectors(id),
  type       TEXT DEFAULT 'residencial'
               CHECK (type IN ('residencial','comercial','terreno','especial')),
  lat        DOUBLE PRECISION,
  lng        DOUBLE PRECISION,
  active     BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 4. VISITAS
CREATE TABLE visits (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  property_id UUID NOT NULL REFERENCES properties(id),
  agent_id    UUID NOT NULL REFERENCES profiles(id),
  status      TEXT NOT NULL CHECK (status IN (
                'visitado_sem_foco',
                'visitado_com_achado',
                'fechado',
                'recusado',
                'nao_localizado',
                'pendente_revisao'
              )),
  notes       TEXT,
  visited_at  TIMESTAMPTZ NOT NULL,
```

```
  synced_at   TIMESTAMPTZ,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- 5. EVIDÊNCIAS (fotos por visita)
CREATE TABLE evidences (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  visit_id     UUID NOT NULL REFERENCES visits(id) ON DELETE CASCADE,
  storage_path TEXT NOT NULL,  -- caminho no Supabase Storage
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

-- 6. LOGS DE ATIVIDADE
CREATE TABLE activity_logs (
  id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id   UUID REFERENCES profiles(id),
  action    TEXT NOT NULL,   -- ex: 'create_visit', 'export_csv'
  entity    TEXT NOT NULL,   -- ex: 'visits', 'properties'
  entity_id UUID,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ========================================
-- RLS POLICIES (Row Level Security)
-- ========================================

ALTER TABLE visits ENABLE ROW LEVEL SECURITY;

-- Agentes vêem apenas suas próprias visitas
CREATE POLICY "agents_own_visits" ON visits
  FOR ALL USING (
    agent_id = auth.uid()
    OR EXISTS (
      SELECT 1 FROM profiles p
      WHERE p.id = auth.uid()
        AND p.role IN ('coordinator','manager','admin')
    )
  );

-- Agentes criam apenas visitas com seu próprio ID
CREATE POLICY "agents_insert_own" ON visits
  FOR INSERT WITH CHECK (agent_id = auth.uid());

-- INDEX para performance
CREATE INDEX idx_visits_property  ON visits(property_id);
CREATE INDEX idx_visits_agent     ON visits(agent_id);
CREATE INDEX idx_visits_status    ON visits(status);
```

```
CREATE INDEX idx_properties_code  ON properties(code);
CREATE INDEX idx_properties_sector ON properties(sector_id);
```

PERMISSÕES POR ROLE

PERMISSÃO	AGENT	COORDINATOR	MANAGER	ADMIN
Vê suas próprias visitas	SIM	SIM	SIM	SIM
Cria / registra visita	SIM	NÃO	NÃO	SIM
Vê painel de cobertura	NÃO	SIM	SIM	SIM
Exporta CSV	NÃO	SIM	SIM	SIM
Gerencia usuários	NÃO	NÃO	PARCIAL	SIM
Importa imóveis CSV	NÃO	NÃO	SIM	SIM

05 Fluxos e Telas

App do agente (Android) e painel do coordenador (Web). Cada tela com suas ações.

📱 APP DO AGENTE — 8 TELAS

1. **Login**
   Campo e-mail + senha. Botão "Entrar". Erro inline se credenciais inválidas. Token JWT salvo no MMKV. Redireciona para Home.

2. **Home / Busca de Imóvel**
   Campo de busca por código ou endereço. Lista de imóveis do setor do agente. Badge de status da última visita. Busca funciona offline (cache MMKV).

3. **Detalhe do Imóvel**
   Código, endereço, tipo. Últimas 5 visitas com status e data. Botão "Registrar Visita" em destaque.

4. **Registrar Visita**

6 botões de status (cores diferentes). Campo de nota opcional. Botão de câmera para foto. Confirmação antes de salvar.

5 **Confirmação de Visita**

Resumo: imóvel, status selecionado, nota, foto (thumbnail). Botões "Confirmar" e "Editar". Salva localmente, enfileira sync.

6 **Histórico Pessoal**

Lista das visitas do agente (hoje, últimos 7 dias). Filtro por status. Contador de visitas realizadas.

7 **Status de Sync**

Número de visitas pendentes de envio. Botão "Sincronizar agora". Indicador visual: verde (sincronizado) / amarelo (pendente) / vermelho (erro).

8 **Configurações**

Nome do agente, setor, versão do app. Botão "Sair". Limpar cache local.

💻 PAINEL DO COORDENADOR — 5 TELAS

1 **Dashboard Principal**

Métricas: total imóveis, % visitados hoje, achados, recusas, pendências. Tabela de agentes ativos com contagem.

2 **Lista de Imóveis**

Tabela filtrável por setor e status. Coluna com última visita e agente. Botão exportar CSV.

3 **Detalhe do Imóvel (Web)**

Histórico completo de visitas com fotos. Timeline visual por data.

4 **Lista de Agentes**

Agentes por setor. Visitas realizadas hoje e no período. Status online/offline.

5 **Exportação**

Filtros: período, setor, status. Download CSV. Nome do arquivo com data.

⚙ ADMIN — 3 TELAS

A **Criar Usuário**

Formulário: nome, e-mail, senha temporária, role, setor. Confirmação por e-mail automática via Supabase Auth.

B **Gerenciar Setores**

Criar e editar setores. Associar imóveis a setor por CSV.

C **Import CSV Imóveis**

Upload de planilha. Preview das primeiras 5 linhas. Confirmar import. Log de erros por linha.

🕐 **Fluxo Crítico do MVP (caminho feliz)**

Login → Buscar imóvel por código → Ver detalhe + histórico → Registrar visita → Selecionar 1 dos 6 status → Tirar foto → Adicionar nota → Confirmar → Salvar localmente → App sincroniza automaticamente ao conectar → Coordenador vê no painel.

06 Passo a Passo de Execução

6 fases do zero ao APK de demo, com estimativas reais de tempo.

FASE	DURAÇÃO	ATIVIDADES	ENTREGÁVEL	RISCO
Fase 0 Definição	3 dias	Criar projeto Supabase, rodar migrations SQL, criar 4 usuários de teste (admin, coord, 2 agentes), importar 50 imóveis seed de Uberlândia, criar repositórios no GitHub	Banco rodando, usuários ativos, imóveis importados	BAIXO
Fase 1 Scaffold + Auth	3 dias	Expo create-app, instalar dependências (supabase-js, MMKV, React Navigation), tela de Login funcional, navegação por role (agente vê app, coord vê rota do painel)	Login funcionando no celular, navegação por role	BAIXO
Fase 2 App do Agente	7 dias	8 telas do app, integração câmera, MMKV offline storage, fila de sync, upload de fotos para Supabase Storage, testes em modo avião	APK de teste com todas as telas funcionando offline	RISCO ALTO
Fase 3 Painel Coordenador	5 dias	Next.js setup, integração Supabase, dashboard com métricas, lista de imóveis filtrável, detalhe com histórico, exportação CSV, deploy Vercel	Painel acessível via URL pública	MÉDIO

FASE	DURAÇÃO	ATIVIDADES	ENTREGÁVEL	RISCO
Fase 4 QA + Seed	4 dias	Seed SQL 200 visitas distribuídas em 30 dias com status variados, APK build via EAS, QA manual completo (10 visitas reais, offline, sync, CSV), correções de bugs	APK final assinado, painel com dados realistas	MÉDIO
Fase 5 Demo / Piloto	1 dia	Roteiro de demo (20 min), 2 celulares Android com APK instalado, painel aberto no notebook, usuários de demo criados, backup em vídeo gravado	Demo pronta para apresentar à prefeitura	BAIXO

⚠ Caminho Crítico

T09 (offline + sync) é a maior trava do projeto. Se o dev travar nessa tarefa, toda a Fase 2 atrasa. Plano B: sync manual por botão "Enviar visitas" — o agente pressiona quando tiver Wi-Fi. Remove a complexidade de sync automático e ainda é usável para demo.

07 Backlog Técnico Atômico

20 tasks com ID, descrição, prioridade, horas e critério de aceite. Total: ~82 horas.

ID	TASK	DESCRIÇÃO	PRIO	HORAS	DEPS	CRITÉRIO DE ACEITE
T01	Setup Supabase	Criar projeto, rodar 6 migrations, configurar RLS, criar 4 usuários de teste	P0	4h	—	Login funciona, RLS bloqueia acesso cruzado
T02	Expo scaffold	create-expo-app, instalar supabase-js, react-navigation, MMKV, EAS config	P0	3h	T01	App abre no Android sem erros
T03	Tela Login	Form e-mail/senha, chamar supabase.auth.signIn, salvar token MMKV, navegar por role	P0	4h	T01, T02	Login válido redireciona, inválido mostra erro inline

ID	TASK	DESCRIÇÃO	PRIO	HORAS	DEPS	CRITÉRIO DE ACEITE
T04	Import CSV imóveis	Script Node.js que lê CSV e faz bulk insert em properties via Supabase API	P0	3h	T01	50 imóveis importados sem erro, códigos únicos
T05	Busca imóvel	Tela Home com campo de busca, lista por setor do agente, funciona com dados em cache	P0	5h	T03, T04	Busca retorna resultados em <500ms offline
T06	Detalhe imóvel	Tela com dados do imóvel e últimas 5 visitas com status e data	P0	4h	T05	Histórico exibe data, status e agente corretamente
T07	Registro visita	6 botões de status, campo nota, confirmação, salva no banco ou fila offline	P0	6h	T06, T09	Visita salva com status, agent_id, property_id e visited_at corretos
T08	Câmera + foto	expo-image-picker, comprimir imagem, salvar path local, upload para Supabase Storage no sync	P1	5h	T07	Foto aparece no detalhe da visita no painel web
T09	Offline + sync CRITICO	MMKV queue, detectar conexão via NetInfo, sync automático ao reconectar, resolver duplicatas por ID	P0	8h	T02	Visita registrada sem internet aparece no painel após reconexão
T10	Setup Next.js	create-next-app, Tailwind, supabase-js, layout base, autenticação coordenador	P0	4h	T01	Painel abre em localhost, login funciona para role coordinator
T11	Dashboard métricas	Cards: total imóveis, visitados hoje, com achado, pendências, recusados. Query SQL agregada.	P0	8h	T10, T16	Números batem com dados do banco, atualiza em tempo real via Supabase Realtime

ID	TASK	DESCRIÇÃO	PRIO	HORAS	DEPS	CRITÉRIO DE ACEITE
T12	Lista imóveis web	Tabela com filtro por setor e status, paginação, última visita	P1	5h	T10	Filtro por setor exibe apenas imóveis do setor selecionado
T13	Lista agentes	Tabela agentes com visitas hoje e no período, agrupado por setor	P1	4h	T10	Contagem de visitas por agente bate com banco
T14	Exportação CSV	Endpoint API Next.js, query com filtros, retorna CSV com headers corretos e encoding UTF-8	P1	3h	T10	CSV abre no Excel com acentos corretos, dados completos
T15	Admin criar usuário	Formulário no painel, chama supabase.auth.admin.createUser via service key	P1	3h	T10	Novo agente consegue fazer login no app mobile
T16	Seed data completo	SQL INSERT 200 visitas distribuídas em 30 dias, 5 agentes, status variados, notas reais	P0	4h	T04	Dashboard mostra métricas realistas para demo
T17	APK build EAS	eas build --platform android --profile preview, testar APK no celular físico	P1	2h	T07, T08, T09	APK instala sem erro e app funciona em modo offline
T18	Detalhe imóvel web	Página com timeline de visitas, fotos, agentes e notas. Navegavel da lista.	P1	4h	T12	Fotos carregam corretamente do Supabase Storage
T19	QA manual	20 visitas reais, teste offline (modo avião), sync, export CSV no Excel, teste em 2 celulares diferentes	P0	4h	T17	Zero crash, sync funciona, CSV abre correto
T20	Deploy Vercel	Conectar repositório, configurar env vars (SUPABASE_URL, ANON_KEY), testar URL de produção	P1	1h	T11	Painel acessível em URL pública, login funciona

Total estimado

~82 horas de desenvolvimento — equivalente a ~2,5 semanas de dev full-time (32h/semana) ou 4–5 semanas part-time (20h/semana). Tasks P0 = 41h, tasks P1 = 41h.

08 O que Founders Fazem Sozinhos

12 tarefas que não precisam de dev, com ferramentas específicas e tempo estimado.

TAREFA	COMO FAZER (FERRAMENTA ESPECÍFICA)	TEMPO	DIFICULDADE	RISCO
Seed data imóveis	ChatGPT: "Gere um CSV com 50 imóveis de Uberlândia-MG, colunas: codigo, endereco, setor, tipo. Setores: Norte, Sul, Leste, Centro."	2h	1/5	BAIXO
Wireframes das telas	Claude: "Descreva o layout mobile da tela de registro de visita com 6 status coloridos, campo de nota e botão de câmera. Use Tailwind classes." Copiar para Excalidraw ou Figma.	3h	2/5	BAIXO
Textos do app	Google Doc com todas as labels, botões, mensagens de erro, mensagens de sucesso e telas vazias. Entregar pro dev como "strings.md".	3h	2/5	BAIXO
Roteiro de demo	Documento de 1 página: passo a passo do que mostrar, ordem das telas, frases de transição, perguntas esperadas da prefeitura com respostas.	2h	1/5	BAIXO
Manual do agente	Canva: PDF de 1 página com screenshots das telas + descrição dos 6 status com exemplos reais (o que é "achado", o que é "recusa").	3h	2/5	BAIXO
QA manual	Registrar 10 visitas reais no app, ligar modo avião, registrar mais 5, reconectar e verificar sync. Exportar CSV e abrir no Excel. Anotar bugs num Google Sheet.	4h	2/5	MÉDIO
Proposta comercial	Adaptar o Dossô v2 para o porte da prefeitura-alvo. Focar em: problema que resolve, métricas do piloto, condições do termo de cooperação, sem promessa de funcionalidades futuras.	3h	3/5	MÉDIO
Gestão do backlog	Notion ou Trello: criar tasks T01–T20 como cards, priorizar P0 no topo, check diário de 30 min com o	1h/semana	2/5	MÉDIO

TAREFA	COMO FAZER (FERRAMENTA ESPECÍFICA)	TEMPO	DIFICULDADE	RISCO
	dev para mover tasks. Sem reuniões longas.			
Seed SQL de visitas	Claude: "Gere um SQL INSERT para 200 visitas na tabela visits, distribuídas em 30 dias, com property_ids UUID da lista abaixo, status variados (70% sem_foco, 15% com_achado, 10% fechado, 5% recusado), notas em português."	2h	2/5	BAIXO
Comunicação com prefeitura	Agendar reunião de 30 min (e-mail + WhatsApp), apresentar proposta de piloto com 1 setor, ~200 imóveis, 2–3 agentes, 30 dias. Pedir termo de cooperação, não contrato pago. Não prometer o que não está pronto.	Contínuo	3/5	ALTO
Planilha CSV imóveis	Google Sheets: criar planilha com colunas exatas (codigo, endereco, setor, tipo). Preencher com dados reais ou gerados. Exportar como CSV UTF-8. Passar pro dev com script T04.	2h	1/5	BAIXO
Definir setores piloto	Junto com a prefeitura: escolher 1 bairro com ~200 imóveis, 2–3 agentes conhecidos, área com histórico de dengue (mais impactante para demo). Documentar o setor escolhido.	2h	2/5	MÉDIO

09 O que Vale Terceirizar

O que contratar, por que, qual perfil buscar e como estruturar milestones.

💰 **Budget e Perfil**

Budget: R$ 8.000–15.000 por projeto (4–5 semanas). Perfil ideal: freelancer React Native + Supabase, portfólio B2B publicado no GitHub, disponível por projeto (não CLT). Aceita pagar por milestone, não por hora.

O QUE TERCEIRIZAR	POR QUE NÃO FAZER SOZINHO	PERFIL NECESSÁRIO	FORMATO	RISCO SE FOUNDERS TENTAREM
Setup Supabase +	RLS policies erradas	Full-stack com Supabase ou	Task única, 4h	ALTO — DADOS VAZAM ENTRE USUÁRIOS

O QUE TERCEIRIZAR	POR QUE NÃO FAZER SOZINHO	PERFIL NECESSÁRIO	FORMATO	RISCO SE FOUNDERS TENTAREM
modelagem (T01)	criam brechas de segurança silenciosas	PostgreSQL		
App agente telas core (T03–T07)	React Native tem quirks de navegação e estado que são lentos de aprender	React Native 2+ anos de experiência	Milestone M1	CRÍTICO — PODE LEVAR 10X MAIS TEMPO
Offline + sync (T09)	Conflict resolution, filas, NetInfo, retry logic — não é trivial	Experiência específica com MMKV ou WatermelonDB	Task dedicada, 8h	CRÍTICO — APP NÃO FUNCIONA NO CAMPO SEM ISSO
Painel coordenador (T10–T14)	Next.js + Supabase Realtime + queries agregadas requerem experiência	Full-stack Next.js + SQL	Milestone M2	MÉDIO — PAINEL FUNCIONA MAS SEM QUALIDADE
Auth + RLS completo	JWT, cookies, session management, policies por role — área de segurança	Full-stack com Supabase Auth	Incluso no M1	ALTO — BRECHAS DE ACESSO ENTRE ROLES
Upload de foto + Storage (T08)	Compresso, permissões da câmera Android, upload com retry	React Native com expo-image-picker	Incluso no M1	MÉDIO — FOTOS NÃO APARECEM OU SEM COMPRESSAÃO

O QUE TERCEIRIZAR	POR QUE NÃO FAZER SOZINHO	PERFIL NECESSÁRIO	FORMATO	RISCO SE FOUNDERS TENTAREM
Build APK EAS (T17)	Configuração de certificate, signing, eas.json — 1h se sabe, 4h se não sabe	Qualquer dev Expo experiente	Task final, 2h	BAIXO — SÓ PERDE TEMPO

Como Contratar Certo

1. Exigir portfólio publicado com pelo menos 1 app React Native no GitHub ou Play Store.
2. Teste técnico de 2 horas: "Crie uma tela de login com Expo + Supabase Auth que salva o token no MMKV."
3. Pagar por milestone, nunca por hora:
- M1 (R$ 5.000): Scaffold + auth + 8 telas app agente com offline
- M2 (R$ 4.000): Painel coordenador + export CSV + deploy Vercel
- M3 (R$ 1.000–2.000): APK EAS + QA + correções

4. Check diário de 30 min via WhatsApp. Sem reuniõo, só status + bloqueios.

10 Cronograma

3 cenários realistas baseados na disponibilidade do dev e no caminho crítico.

CENÁRIO REALISTA

8 semanas

Dev part-time, 4h/dia, 20h/semana. Sem imprevistos críticos. Recomendado para primeiro projeto com dev novo.

CENÁRIO ACELERADO

4 semanas

Dev full-time, 8h/dia, 40h/semana. Dev experiente com Expo + Supabase. Possível, exige foco total.

CENÁRIO CONSERVADOR

12 semanas

Dev part-time com imprevistos. T09 (offline) travou 1 semana, 1 retrabalho de design. Realidade para a maioria.

SEMANA	REALISTA (8 SEM)	ACELERADO (4 SEM)	CONSERVADOR (12 SEM)
Semana 1	T01 + T02 + T03 — Supabase, scaffold, login	T01 + T02 + T03 + T04 + T05	T01 + T02 — Setup apenas
Semana 2	T04 + T05 + T06 — Imóveis, busca, detalhe	T06 + T07 + T08 — Visita + foto	T03 + T04 — Login + import CSV
Semana 3	T07 + T08 — Registro visita + foto	T09 + T10 — Offline + scaffold Next.js	T05 + T06 + T07 — Busca + detalhe + visita
Semana 4	T09 — Offline + sync (semana crítica)	T11 + T12 + T14 — Dashboard + CSV	T08 + T09 — Foto + offline (semana pesada)
Semana 5	T10 + T11 — Next.js + dashboard	T15 + T16 + T17 — Admin + seed + APK	Buffer / correções T09
Semana 6	T12 + T13 + T14 + T15 — Listas + CSV + admin	T19 + T20 — QA + deploy (DEMO PRONTA)	T10 + T11 — Next.js + dashboard
Semana 7	T16 + T17 + T18 — Seed + APK + detalhe	—	T12 + T13 + T14 + T15 — Listas + CSV
Semana 8	T19 + T20 — QA + deploy (DEMO PRONTA)	—	T16 + T17 + T18 — Seed + APK + detalhe
Semanas 9–12	—	—	T19 + T20 + QA + correções (DEMO PRONTA)

⚠ Caminho Crítico

T09 (offline/sync) → T11 (dashboard) → T16 (seed) → T19 (QA) → T20 (deploy). Qualquer atraso em T09 propaga para tudo. **Plano B imediato:** se T09 não ficar pronto em 8 horas, mudar para

sync manual por botão. Isso resolve o bloqueio e ainda é demonstrável.

11 Prompts para Subagentes

7 prompts prontos para usar no Claude, ChatGPT ou Cursor para acelerar a execução.

📊 1. Modelagem do Banco (Supabase Schema Completo)

```
Você é um engenheiro de banco de dados especialista em Supabase e PostgreSQL.

Crie o schema SQL completo para o ZoonoseApp MVP com as seguintes especificações:
- 6 tabelas: profiles, sectors, properties, visits, evidences, activity_logs
- profiles extende auth.users via FK
- visits tem 6 status: visitado_sem_foco, visitado_com_achado, fechado, recusado, nao_localiza
- RLS habilitado em todas as tabelas
- Policy: agentes veem apenas suas próprias visitas, coordinators/managers/admins veem todas
- Indexes de performance nas colunas mais consultadas
- DEFAULT para municipality = 'Uberlândia'
- Inclua seed SQL com 4 usuários de teste e 2 setores

Retorne o SQL completo pronto para rodar no Supabase SQL Editor.
```

📱 2. UI/UX das 8 Telas Mobile

```
Você é um designer de produto especialista em apps mobile B2G (governo).

Descreva o layout detalhado das 8 telas do app do agente do ZoonoseApp:
1. Login, 2. Home/Busca Imóvel, 3. Detalhe Imóvel, 4. Registrar Visita,
5. Confirmação, 6. Histórico Pessoal, 7. Status Sync, 8. Configurações

Para cada tela, especifique:
- Componentes React Native (FlatList, TextInput, TouchableOpacity, etc.)
- Layout e hierarquia visual
- Cores e estados (hover, disabled, loading)
- Casos de erro e telas vazias

O app usa tema escuro, fundo #0a1628, azul #3b82f6, verde #10b981.
```

```
A tela 4 (Registrar Visita) tem os 6 status como botões grandes coloridos.

Retorne em formato de especificação técnica para o desenvolvedor.
```

🔒 3. App Agente — Tela de Registro com Offline

```
Você é um desenvolvedor React Native especialista em Expo e Supabase.

Implemente a tela "Registrar Visita" do ZoonoseApp com:
- 6 botões de status: visitado_sem_foco, visitado_com_achado, fechado, recusado, nao_localizad
- Cada status com cor específica (verde, vermelho, cinza, laranja, amarelo, azul)
- Campo de nota opcional (TextInput multiline)
- Botão de câmera usando expo-image-picker
- Ao confirmar: salvar no MMKV (offline) OU inserir direto no Supabase se online
- Fila de operações pendentes no MMKV, sync automático ao detectar conexão via @react-native-c
- TypeScript com tipos corretos para as entidades

Retorne o código completo do componente RegisterVisitScreen.tsx e o hook useOfflineSync.ts.
```

📈 4. Dashboard do Coordenador

```
Você é um desenvolvedor Next.js + Supabase especialista em dashboards operacionais.

Implemente o dashboard do coordenador do ZoonoseApp em Next.js 14 com App Router:
- Cards de métricas: total de imóveis, % visitados hoje, visitados com achado, fechados/recusa
- Tabela de agentes com visitas realizadas hoje e nos últimos 7 dias
- Filtros por setor e período (hoje, 7 dias, 30 dias)
- Atualização em tempo real via Supabase Realtime
- Botão de exportação CSV (endpoint API Route)
- Tailwind CSS com tema escuro (#0a1628)
- Queries SQL otimizadas com GROUP BY e COUNT

Retorne os arquivos: app/dashboard/page.tsx, components/MetricCard.tsx, app/api/export/route.t
```

✅ 5. QA Checklist Completo

```
Você é um QA engineer especialista em apps mobile e sistemas offline-first.

Crie um checklist completo de QA para o ZoonoseApp MVP cobrindo:

FLUXOS CRÍTICOS:
1. Login com credenciais válidas e inválidas
2. Busca de imóvel por código e endereço
3. Registro de visita com cada um dos 6 status
4. Captura de foto e verificação no painel web
5. Modo offline: registrar 5 visitas sem internet, reconectar, verificar sync
6. Exportação CSV e abertura no Excel com acentos corretos
7. Permissões: agente não consegue acessar painel de coordenador
8. RLS: agente A não vê visitas do agente B

Para cada item: passos, resultado esperado, resultado obtido, status (PASS/FAIL).
Formato de planilha Google Sheets.
```

💾 6. Seed Data de Demo (200 visitas)

```
Você é um engenheiro de dados especialista em SQL e PostgreSQL.

Gere um SQL com INSERT de 200 visitas na tabela visits do ZoonoseApp com:
- Distribuídas em 30 dias (de hoje para trás)
- 5 agentes diferentes (UUIDs que vou fornecer)
- 50 imóveis diferentes de Uberlândia-MG (UUIDs que vou fornecer)
- Distribuição de status: 65% visitado_sem_foco, 15% visitado_com_achado, 10% fechado, 5% recu
- Notas em português realistas (ex: "Caixa d'água com larvas", "Imovél vazio, quintal com entu
- Concentração maior nas últimas 2 semanas (mais realista)
- visited_at com horários entre 07:00 e 17:00 (horário de trabalho)

Retorne o SQL completo pronto para rodar no Supabase SQL Editor.
```

📄 7. Documentação de Setup (README Técnico)

```
Você é um tech writer especialista em documentação de projetos open source.

Crie o README.md técnico do ZoonoseApp MVP com:
- Descrição do projeto (1 parágrafo)
- Arquitetura: diagrama em texto (Expo App + Next.js Painel → Supabase)
```

```
  - Pré-requisitos: Node 20, Expo CLI, conta Supabase
  - Setup passo a passo:
    1. Clonar repositórios (monorepo ou 2 repos)
    2. Configurar .env com SUPABASE_URL e ANON_KEY
    3. Rodar migrations SQL no Supabase Dashboard
    4. npm install em /app e /web
    5. npx expo start para o app
    6. npm run dev para o painel
    7. eas build para gerar APK
  - Estrutura de diretórios comentada
  - Variáveis de ambiente necessárias
  - Como criar usuários de teste
  - Como importar imóveis via CSV

  Linguagem técnica mas acessível, em português brasileiro.
```

12 Critérios de Demo para a Prefeitura

Roteiro de 20 minutos, o que mostrar, o que não pode falhar e o que esconder.

MOMENTO	DURAÇÃO	O QUE FAZER	SCRIPT SUGERIDO
1. Contexto	2 min	Mostrar o painel web já aberto com dados reais de Uberlândia. Não começar pelo app ainda.	"Veja que temos 50 imóveis do bairro X com 200 visitas nos últimos 30 dias. Esse é o tipo de visibilidade que vocês têriam em tempo real."
2. App do Agente	8 min	Pegar o celular, abrir o app, buscar imóvel UBL-0042, mostrar histórico, registrar uma visita AO VIVO com foto e nota. Ligar modo avião e registrar mais uma.	"O agente chega no imóvel, busca pelo código, vê que já foi visitado em janeiro. Registra o status, tira uma foto da caixa d'água. Olha — até sem internet funciona."
3. Painel do Coordenador	5 min	Voltar para o notebook. Mostrar que a visita registrada já apareceu no painel (reconectar o celular antes). Clicar no agente, mostrar histórico. Exportar CSV e abrir no Excel.	"O coordenador viu agora mesmo que o agente Carlos registrou uma visita com achado. Clica aqui — vê a foto. E eu posso exportar isso para o Excel em 2 segundos."
4. Pergunta-Chave	3 min	Perguntar ativamente: "Quais setores do município vocês gostariam de	"Para um piloto, a gente precisaria de 1 setor, ~200 imóveis e 2 a 3 agentes

MOMENTO	DURAÇÃO	O QUE FAZER	SCRIPT SUGERIDO
		começar? Quantos agentes são alocados por setor?" Anotar as respostas.	por 30 dias. Qual área faria mais sentido começar?"
5. Próximo Passo	2 min	Propor termo de cooperação (não contrato pago). Deixar APK instalado no celular do coordenador antes de sair.	"Não estamos pedindo orçamento agora. Só queremos um termo de cooperação para validar com dados reais. Vocês ganham visibilidade, a gente ganha validação."

✅ O que Parece Sofisticado mas É Simples

- Dashboard com números grandes — são 4 queries SQL com COUNT
- Visita em tempo real no painel — é Supabase Realtime (1 linha de código)
- Foto no detalhe do imóvel — é uma URL pública do Supabase Storage
- Exportação CSV "completa" — é uma query SQL com download
- Modo offline — é MMKV + fila simples (com sync manual é ainda mais fácil)
- Histórico do imóvel — é um JOIN com ORDER BY created_at

❌ O que NãO Pode Falhar na Demo

⚠ Login — ensaiar até ser automático

⚠ Foto — testar no celular que vai ser usado

⚠ Sync offline — testar modo avião antes da reunião

⚠ CSV abrindo no Excel — testar encoding com acentos

⚠ Internet da sala da prefeitura — levar hotspot próprio

⚠ Bateria dos celulares — chegar carregado, levar carregador

👁 O que ESCONDER na Demo

Não mostrar: tela de admin de usuários (parece incompleta), import CSV (interface crua), configurações do app (versão de dev visível), qualquer loading de mais de 2 segundos (seed os dados antes). **Dica de backup:** gravar vídeo do fluxo completo funcionando. Se algo falhar na demo ao vivo, passar o celular e dizer "já gravamos o fluxo aqui" e mostrar o vídeo.

13 O que NãO Fazer Agora

15 erros comuns de founders técnicos que atrasam o MVP sem adicionar valor real.

---

1. Offline perfeito antes de validar adoção

Conflict resolution distribuído é engenharia de nível Notion/Linear. Para MVP, sync manual por botão é suficiente e demonstrável. Offline perfeito só depois que a prefeitura assinar.

2. Mapas e heatmaps

Bonito na apresentação, inútil sem dados reais de GPS coletados em campo. Adiciona 2 semanas de desenvolvimento e não muda a decisão da prefeitura no piloto.

3. App iOS agora

Prefeituras brasileiras usam Android. iOS exige Mac + Apple Developer Account ($99/ano) + processo de review. Adicione depois de ter contrato.

4. Play Store antes do piloto

APK via link direto é mais rápido, flexível e suficiente para piloto. Play Store tem review de 3–7 dias e requer conta de desenvolvedor. Publique só na versão comercial.

5. Notificações push

Requer Firebase Cloud Messaging, permissões do usuário, lógica de gatilho. Para o piloto, o coordenador olha o painel. Push notification é conforto, não necessidade do MVP.

6. Multi-tenant desde o início

Adicionar organização como camada de isolamento triplica a complexidade do schema e da RLS. Um cliente primeiro, depois multi-tenant. Não tem como vender para 2 prefeituras ao mesmo tempo no piloto.

7. Dashboard "bonito" com gráficos

Recharts, Chart.js, D3 — adicionam complexidade sem aumentar a decisão da prefeitura. Números grandes em cards convencem mais que gráficos de linha em MVP. Gráficos depois de ter dados reais de 90 dias.

8. Contratar designer antes de validar

Design system, biblioteca de componentes, Storybook — custo alto antes de saber se o produto tem adoção. Use shadcn/ui ou Tailwind direto. Design profissional na versão paga.

9. Firebase em vez de Supabase

Firebase Firestore não tem SQL real, o que torna queries agregadas (métricas do dashboard) muito mais complexas. Supabase é PostgreSQL — qualquer dev conhece SQL. Não mude a stack por preferência.

10. Backend customizado Node/Express

Supabase Edge Functions e RLS eliminam a necessidade de backend customizado para o MVP. Adicionar Express/NestJS é mais uma coisa para manter, testar e deployar. YAGNI.

11. Montar time antes de ter cliente

Designer + product manager + segundo dev antes do primeiro contrato é burn de caixa sem retorno. Um dev freelancer por projeto é o modelo certo até ter receita recorrente.

12. Billing no sistema

Stripe, assinatura, plãos, invoices dentro do produto — o piloto é gratuito por definição. Invoice manual via PDF ou Conta Azul até ter 3 clientes pagantes. Não construa billing antes de ter alguém para cobrar.

13. Reuniões longas sem código na tela

Qualquer reunião de mais de 30 minutos sobre o produto que não termina com uma decisão ou uma tarefa é tempo perdido. Check diário de 15 min com o dev. Reunião semanal de 30 min de retrospectiva. Nada mais.

14. Relatórios PDF automáticos

Puppeteer, html-pdf, LaTeX — relatório PDF é V2. A prefeitura aceita CSV no Excel para o piloto. PDF bonito impressiona, mas não é critério de aceite do piloto. Adicione quando pedirem.

15. Esperar o produto "estar pronto"

O produto nunca vai estar pronto. Demo com APK funcionando e 50 imóveis seed é suficiente para ter a primeira conversa real. A validação não acontece no Figma — acontece com um agente real em campo.

14 Recomendação Final

O caminho exato que eu faria como CTO temporário desse produto.

"Se eu estivesse executando isso sozinho como CTO temporário, eu faria exatamente assim:"

Plano baseado em produto B2G, MVP de 4–8 semanas, sem recursos de equipe grande.

1 **Semana 1 — Contratar e Ativar**

Contratar dev freelancer por R$ 10.000 em 3 milestones. Criar o projeto no Supabase e rodar o schema SQL completo no mesmo dia. Criar os 4 usuários de teste (admin, coordenador, 2 agentes). Importar 50 imóveis reais de Uberlândia. Antes do fim da semana: ver o login funcionando no celular do dev. Se não funcionar até sexta, o dev está com problema.

2

Semanas 2–3 — Paralelismo

Check de 30 minutos por dia com o dev (WhatsApp). Enquanto o dev constrói as telas: escrever todos os textos do app (labels, erros, mensagens de sucesso), criar o roteiro de demo de 20 minutos, escrever o manual do agente em PDF (1 página, Canva), gerar seed de 200 visitas com SQL via Claude, preparar a proposta de piloto adaptada para a prefeitura-alvo. Nenhuma dessas tarefas precisa do dev. Não deixe a semana passar sem produzir.

3

Semana 4 — QA Brutal

Pegar 2 celulares Android. Instalar o APK nos dois. Registrar 20 visitas reais. Testar modo avião: desligar a internet, registrar 5 visitas, religar, verificar se sincronizou no painel. Exportar o CSV e abrir no Excel — verificar se os acentos estão corretos. Mostrar para uma pessoa que nunca viu o produto (não pode ser o dev nem os founders). Observar o que ela trava. Corrigir. Gerar o APK final com EAS Build.

4

Semana 5 — Demo e Piloto

Agendar reunião de 30 minutos com a prefeitura amiga (a que já demonstrou interesse). Chegar com o painel aberto, 2 celulares carregados, hotspot próprio e o vídeo de backup gravado. Mostrar o que tem. Ser honesto sobre o que ainda não está pronto. Propor um piloto com termo de cooperação — não contrato, não pagamento, só dados reais. Deixar o APK instalado no celular do coordenador antes de sair.

🎯 **A Única Coisa que Importa**

Uma prefeitura disposta a assinar um termo de cooperação. MVP não precisa estar perfeito — precisa ser honesto e crivél. Menos promessa. Mais prova.

Se a prefeitura não assinar depois da demo, o problema não é o produto — é o relacionamento ou a dor não é suficientemente urgente para aquela pessoa. Nesse caso: mude o interlocutor ou a prefeitura, não o produto.

ESTRUTURA DE DIRETÓRIOS DO PROJETO

```
zoonoseapp/
├── app/                          # Expo React Native (agente)
|   ├── src/
|   |   ├── screens/
|   |   |   ├── LoginScreen.tsx
|   |   |   ├── HomeScreen.tsx
|   |   |   ├── PropertyDetailScreen.tsx
|   |   |   ├── RegisterVisitScreen.tsx
|   |   |   ├── ConfirmVisitScreen.tsx
|   |   |   ├── HistoryScreen.tsx
|   |   |   ├── SyncStatusScreen.tsx
```

```
│   │   │   └── SettingsScreen.tsx
│   │   ├── hooks/
│   │   │   ├── useOfflineSync.ts   # T09 - fila MMKV + sync
│   │   │   ├── useAuth.ts
│   │   │   └── useProperties.ts
│   │   ├── lib/
│   │   │   ├── supabase.ts         # client Supabase configurado
│   │   │   └── mmkv.ts             # instância MMKV
│   │   └── types/
│   │       └── index.ts            # Visit, Property, Profile, etc.
│   ├── app.json
│   ├── eas.json
│   └── package.json
│
├── web/                          # Next.js (painel coordenador)
│   ├── app/
│   │   ├── (auth)/
│   │   │   └── login/page.tsx
│   │   ├── dashboard/
│   │   │   ├── page.tsx            # T11 - métricas
│   │   │   ├── imoveis/page.tsx    # T12 - lista
│   │   │   ├── agentes/page.tsx    # T13 - lista agentes
│   │   │   └── imoveis/[id]/page.tsx # T18 - detalhe
│   │   └── api/
│   │       └── export/route.ts     # T14 - CSV download
│   ├── components/
│   │   ├── MetricCard.tsx
│   │   ├── AgentTable.tsx
│   │   └── PropertyTable.tsx
│   ├── lib/
│   │   └── supabase.ts
│   └── package.json
│
├── supabase/
│   ├── migrations/
│   │   ├── 001_schema.sql          # 6 tabelas + RLS
│   │   ├── 002_indexes.sql
│   │   └── 003_seed.sql            # 50 imóveis + 200 visitas
│   └── config.toml
│
└── scripts/
    ├── import-imoveis.js           # T04 - CSV import
    └── seed-visitas.sql            # T16 - 200 visitas demo
```

ZoonoseApp

MVP Plano CTO — Documento de Execução

Stack: Expo + Supabase + Next.js • Prazo: 4–8 semanas • Budget: R$ 8.000–15.000

Gerado em Março 2026 — Ubelândia, MG